

FFT Accelerator - Midterm Presentation Speech Notes

Group 9	18-3-2026	~15-20 minutes
---------	-----------	----------------

Slide 1 (Page 1) - "FFT Accelerator"

- Good morning everyone, we are Group 9 and today we will present our progress on the FFT accelerator project.
- We will cover two main parts: high performance optimizations and energy-efficient design strategies.

Slide 2 (Page 2) - Section Divider: "Part I - High Performance"

- Let us start with Part I, where we focus on maximizing performance. This work was led by Shanghong, Alessandro, Yaonan, Leonardo, and Anastasis.

Slide 3 (Page 3) - Section Divider: "01 - Baseline Arch & Profiling"

- First, we need to understand what we started with before we can improve it.

Slide 4 (Page 4) - "An overview of given files"

- Here is the SoC we are working with. At its core is a PicoRV32 processor running the RV32I instruction set, connected over a 32-bit bus to Flash, SRAM, UART, GPIO, and our accelerator block.
- The memory map spans from on-chip SRAM at address zero up to user peripherals at 0x03FFFFFF, and our accelerator lives in that user peripheral space.
- On the RTL side, the key files are et4351.v for the top-level chip, picorv32.v for the CPU, and then accelerator.v, accelerator_fft.v, and accelerator_mem.v for our design space.
- The important constraint to note is the 1 kB SRAM and 4 MB Flash -- memory is very limited, which shapes every decision we make.

Slide 5 (Page 5) - "Radix-2 algorithm characteristics"

- Our baseline implements a radix-2 decimation-in-time FFT. For $N = 2^m$ points, we have $\log_2(N)$ stages, $N/2$ unique twiddle factors, and $N/2$ in-place butterflies per stage.
- The diagram shows an 8-point example with bit-reversed input ordering feeding into the butterfly network. Our actual design runs a 32-point FFT, so $N = 32$, giving us 5 stages with 16 butterflies each.
- The in-place property is key -- each butterfly reads two values, computes, and writes back to the same locations, which means we do not need extra buffer memory.

Slide 6 (Page 6) - "Baseline overview"

- Now let us look at the cycle budget. A single butterfly takes 9 cycles: 4 reads for the real and imaginary parts of both inputs, 1 compute cycle, then 4 writes for the outputs.
- Per stage, we load 2 twiddle factor words, then execute 16 butterflies at 9 cycles each, giving 146 cycles per stage.
- Across all 5 stages, plus 1 cycle each for INIT and FINISH, the total is 732 cycles. At 12 MHz, that is about 61 microseconds for one 32-point FFT.

Slide 7 (Page 7) - "Bottleneck of the baseline"

- When we profile those 732 cycles, the breakdown is striking: 650 cycles -- 88.8% -- are pure memory access. Only 80 cycles, about 11%, are actual computation. Control overhead is negligible at 0.3%.

- The root cause is the scalar memory port bottleneck. Each butterfly touches 8 memory words but computes in just 1 cycle, so the datapath is starved waiting for data.
 - There is also no pipelining and no data reuse between butterflies, so every value is read from and written back to SRAM individually. This tells us exactly where to attack.
-

Slide 8 (Page 8) - Section Divider: "02 - Exploration Journey"

- With that bottleneck identified, let us walk through our four design explorations.
-

Slide 9 (Page 9) - "Exploration 1: Register File Architecture"

- Our first idea was simple: if memory access is the bottleneck, stop accessing memory during computation. We introduced a register file with 64 data registers and 10 twiddle registers.
 - We front-load all SRAM reads into registers before computation begins, then the butterfly operates entirely on registers with no memory stalls. After all stages, we store results back.
 - The FSM simplifies from the 9-state interleaved design to a clean 6-state flow: INIT, LOAD_TW, LOAD_DATA, COMPUTE, STORE, FINISH. The butterfly itself becomes single-cycle combinational.
 - This gives us 3.33x speedup, going from 732 down to 220 cycles -- about 18.3 microseconds at 12 MHz -- and it is a drop-in replacement requiring zero firmware changes.
-

Slide 10 (Page 10) - "Exploration 2: 2x Parallel Butterflies"

- Next we asked: can we do two butterflies at once? We added a second independent butterfly datapath running in parallel.
 - The COMPUTE phase is now split into a FILL sub-phase and a BUTTERFLY sub-phase. With two lanes, we process $N/2$ butterflies in pairs, so we need only $N/4$ cycles per stage.
 - We reached 206 cycles total, a 3.55x speedup over baseline. However, we noticed a new bottleneck: the serial twiddle fill, where each twiddle is computed as $tw[k] = tw[k-1] \times prim[stage]$, takes 26 of the 66 compute cycles.
 - Area grew to 308K gates, with the accelerator occupying 71.3% of the SoC. The insight here is that the twiddle fill overhead becomes the next target.
-

Slide 11 (Page 11) - "Exploration 3: Software Twiddle Preload"

- This is where hardware-software co-design really paid off. Instead of computing twiddle factors in hardware, we moved that to firmware.
 - The CPU pre-computes all $N/2 = 16$ twiddle factors in Q12 fixed-point and writes them to 32 CSR registers before enabling the accelerator. This eliminates the LOAD_TWIDDLE state and the FILL sub-phase entirely.
 - The FSM simplifies to just 5 states. SRAM requirements drop from 128 to 64 words, cutting memory area by 50.5%. Total SoC area dropped 15.6% from 308K to 260K.
 - We reached 170 cycles, a 4.31x speedup. The key insight is that a single-shot Q12 quantization on the CPU is more efficient and more accurate than chained multiplications in hardware.
-

Slide 12 (Page 12) - "Exploration 4: Pipelined Butterfly Datapath"

- Our biggest leap came from pipelining. We split the combinational butterfly into a 4-stage micro-pipeline: FETCH, MUL1, MUL2, ADD.
 - By cutting the combinational depth to roughly one quarter, we can clock the accelerator at approximately 60 MHz -- 5 times the baseline frequency.
 - With 2 parallel lanes and single-cycle throughput, a new butterfly pair enters the pipeline every cycle, giving us 55 compute cycles. Total is 185 cycles, but at 5x the clock, so effective latency drops to about 3.1 microseconds.
 - That is roughly a 20x speedup over the 61-microsecond baseline. The factor is 3.96x fewer cycles times 5x frequency. The remaining constraint is memory bandwidth -- we cannot feed the pipeline any faster.
-

Slide 13 (Page 13) - Section Divider: "03 - Roadmap"

- Now let us look at what comes next for the high-performance track.
-

Slide 14 (Page 14) - "Further improvements"

- We are exploring running the accelerator on a faster clock domain, potentially with dual-edge clocking, to double the effective bandwidth from registers to logic.
 - We also want to investigate where the optimal stopping point is -- at what point does additional hardware complexity stop yielding meaningful gains.
 - On the physical design side, we need to explore what synthesis and place-and-route settings give us the best power-performance-area tradeoffs.
 - These are open questions we will address in the back-end phase.
-

Slide 15 (Page 15) - "Next steps"

- Our timeline is straightforward. This week, Week 6, we close the front-end RTL. Week 7 we run the back-end flow -- synthesis, place and route, optimization. Week 8 is reserved for testing, verification, and writing the final report.
 - We are on track and confident in this schedule.
-

Slide 16 (Page 16) - Section Divider: "Part II - Energy Efficient"

- Now let us switch to Part II, where the goal shifts from raw speed to minimizing energy per FFT.
-

Slide 17 (Page 17) - "Energy Saving Ideas"

- We have identified several energy-saving strategies. These include mixed radix-2/radix-4, real-valued FFT optimization, clock gating, register offloading, trivial-twiddle fast paths, removing recursive twiddle updates, and LUT-based twiddle replacement.
 - We will walk through each of the major ones now.
-

Slide 18 (Page 18) - "Radix 2 and 4 Mix"

- By mixing radix-4 and radix-2 stages, we reduce the number of stages from 5 down to 3. Fewer stages means fewer memory passes and less switching activity.
 - The multiplication count drops from 80 to 64 per FFT. Latency also improves since we are doing less total work.
 - The tradeoff is hardware complexity: a radix-4 butterfly unit has 4 inputs instead of 2, so the datapath is wider. But the energy savings from fewer stages and fewer multiplications outweigh the added area.
 - Overall, less time active means less dynamic power, which is exactly what we want for energy efficiency.
-

Slide 19 (Page 19) - "Real FFT (RFFT) Optimization"

- Our input data is real-valued, but the baseline runs a full 32-point complex FFT where all imaginary inputs are zero -- that is wasted computation and wasted memory.
 - The RFFT trick packs 32 real samples into 16 complex values, runs a 16-point complex FFT, then applies a post-processing recombination step to recover the 17 unique frequency bins. The other 15 bins follow from conjugate symmetry: $X[k] = X^*[32-k]$.
 - This halves the FFT size, reducing arithmetic, memory usage, and area. We can combine this with 2 radix-4 stages plus post-processing for maximum efficiency.
-

Slide 20 (Page 20) - "Trivial-Twiddle Fast Paths (1)"

- In every FFT, a significant number of twiddle factors are trivial values: +1, -1, +j, or -j. The baseline still runs a full complex multiplication for these cases, which is wasteful.
 - When the twiddle is +1, the output equals the input. For -1, we just negate. For +j, we swap real and imaginary with a sign flip: +j times $(a + jb)$ equals $-b + ja$. For -j, it is $b - ja$.
 - These cases require only sign flips and swaps, no actual multiplier activity.
-

Slide 21 (Page 21) - "Trivial-Twiddle Fast Paths (2)"

- Importantly, adding these fast paths does not change latency at all -- the FSM, states, and cycle count stay the same.
 - The benefit is purely in energy: the multiplier does not toggle, internal datapath switching is reduced, and dynamic power drops for every trivial-twiddle butterfly.
 - Since roughly a quarter to a third of all twiddle factors in a 32-point FFT are trivial, this is a meaningful reduction in energy per transform.
-

Slide 22 (Page 22) - "Removing Recursive Twiddle Update (1)"

- In the baseline, twiddle factors are generated recursively: w is updated as w times w_m each iteration. This requires an extra complex multiplication just for twiddle progression, not for actual FFT computation.
 - We can replace this with a precomputed hardcoded lookup table indexed by stage m and butterfly index k . The FFT computation itself is completely unchanged.
 - This eliminates the twiddle-generation arithmetic path entirely and replaces it with a simple table read.
-

Slide 23 (Page 23) - "Removing Recursive Twiddle Update (2)"

- Again, latency stays the same -- same FSM, same number of states and cycles.
 - But the energy impact is clear: the extra complex multiply is gone, fewer registers toggle on each cycle, and the switching activity in the twiddle-generation logic drops significantly.
 - This is a clean win with no performance penalty.
-

Slide 24 (Page 24) - "No Twiddle Memory Reads (LUT based) (1)"

- The baseline reads the primitive twiddle factor W_m from memory using two dedicated states: $READ_W_M_RE$ and $READ_W_M_IM$.
 - But $W_m = e^{(-j2\pi/m)}$ depends only on the stage index m , which takes just 5 possible values for our 32-point FFT: m equals 2, 4, 8, 16, 32. These are constants.
 - We can hardcode these 5 complex values as stage-local constants and skip the memory reads entirely.
-

Slide 25 (Page 25) - "No Twiddle Memory Reads (LUT based) (2)"

- The rest of the butterfly flow is unchanged, so latency does not improve since those $READ_W_M$ states are still present in the FSM.
 - However, the memory read activity is eliminated -- no address generation, no data bus toggling, no SRAM read energy for those cycles.
 - The expected outcome is the same runtime but measurably lower dynamic energy from reduced switching on the memory and read paths.
-

Slide 26 (Page 26) - "Thank you for your attention!"

- That concludes our presentation. To summarize: on the performance side, we achieved roughly 20x speedup through register files, parallelism, software twiddle preload, and pipelining. On the energy side, we have a clear set of strategies targeting unnecessary multiplications, memory accesses, and switching activity.
 - We are happy to take any questions. Thank you.
-

Slide 27 (Page 27) - Blank

- (No speech needed -- backup slide.)